

RunPulse.

Marketing analytics, forecasting and data-quality platform
for a mass-participation running event

Project	RunPulse Analytics
Live	runpulse-bhargavi-aitwade-mit-wpu-capstone.vercel.app
Repository	github.com/shashanx52/runpulse-analytics
Stack	Next.js 14 · TypeScript · Recharts · Python / scikit-learn
Scope	13 views · 76,453 fact rows · 6 trained models
Document	Project guide for a first-time reader

Contents

1. **What the project is** — the problem it answers
2. **How to run it** — three commands
3. **Technology map** — what is used where, and why
4. **How data flows** — from simulation to screen
5. **The dataset** — files, grain, and what the numbers look like
6. **The thirteen views** — each one, with a screenshot and what it is for
7. **The models** — six of them, with real measured accuracy
8. **Data quality** — the checks, and two faults the system has to catch
9. **Engineering decisions** — four problems and how they were solved
10. **File map** — what every folder and key file does
11. **Limitations** — what this does not prove

1 · What the project is

A working analytics platform for the marketing team of a large city running event, covering a 172-day registration campaign across four paid channels and four earned ones.

An event marketing team has one budget and four weekly questions. Every part of this application maps onto one of them:

THE QUESTION	WHERE IT IS ANSWERED
Where did the money go, and what came back?	Overall, Marketing Mix, and the four channel views
Which channel should get the next rupee?	ML Lab — spend response curves and the budget optimiser
Are these numbers even correct?	Data Quality — reconciliation and fault detection
How does the season finish?	Forecast — projection to race day with intervals

The event is the **Velocity Run Series 2026**: eight ticket types from a 2 km children's run to a full marathon, sold across six Indian cities, with race day on 23 August 2026. Media runs on Meta, Google, LinkedIn and newspaper print, alongside organic search, email, referral and affiliate traffic.

Scale of the thing

VIEWS	13	FACT ROWS	76,453
Trained models	6	Session records	79,996
TypeScript files	49	Reporting levels	22
API endpoints	8	Campaign season	172 days

2 · How to run it

```
git clone https://github.com/shashanx52/runpulse-analytics
cd runpulse-analytics

npm install                # install dependencies
npm run gen:data           # build the dataset      ~15 seconds
pip install -r ml/requirements.txt  # numpy, pandas, sklearn
npm run train              # fit the six models      ~60 seconds
npm run dev                # http://localhost:3100
```

The application runs with only `npm run gen:data` . The training step is needed by two views — Forecast and ML Lab — and everything else works without Python installed at all.

The AI Analyst view

The AI Analyst needs a Google Gemini API key. Create `.env.local` in the project root:

```
GEMINI_API_KEY=your_key_here
```

Without a key that one view shows a clear message and the other twelve are unaffected. The key is read server-side only and is never sent to the browser.

Every npm script

COMMAND	WHAT IT DOES
<code>npm run dev</code>	Development server with hot reload, port 3100
<code>npm run build</code>	Production build
<code>npm run start</code>	Serve the production build
<code>npm run typecheck</code>	TypeScript check, no emit
<code>npm run gen:data</code>	Run the data simulation, writes <code>data/*.csv</code>
<code>npm run train</code>	Fit the models, writes <code>ml/models/bundle.json</code>

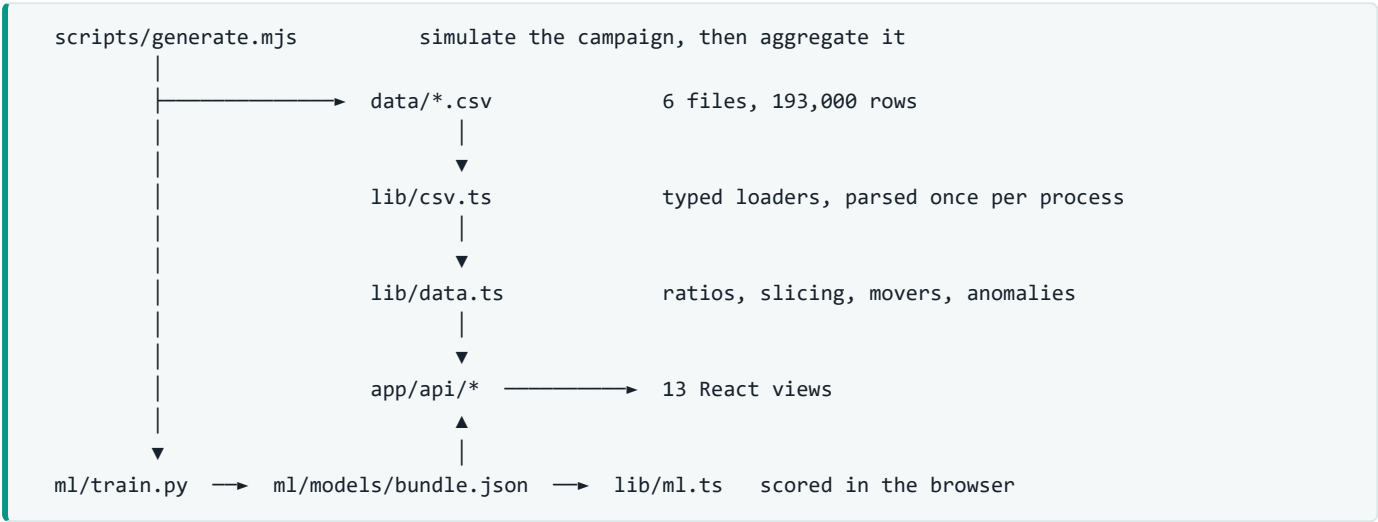
3 · Technology map — what is used where

Every technology in the project, the exact job it does, and why it was chosen over the obvious alternative.

TECHNOLOGY	USED FOR	WHY THIS ONE
Next.js 14 App Router	Page shell, the eight API route handlers, production build	One framework serves both the UI and the server endpoints, so the typed row shapes are shared between them instead of being redeclared on each side.
TypeScript strict mode	All 49 source files	Every table column and chart series is keyed by a field name. Strict typing turns a renamed field into a build error instead of a blank column nobody notices.
React 18	The 13 views and 10 shared components	Views share one context object and recompute with <code>useMemo</code> , so switching a filter re-derives in memory with no server round trip.
Recharts	Every chart: bars, lines, stacked areas, prediction bands	Composes into React and reads colours from the live theme, so all three themes work without a second chart config.
Papaparse	Reading the CSV files server-side	Handles quoted fields and embedded commas correctly. A hand-written split on commas breaks the first time a campaign name contains one.
Python 3.10 numpy, pandas	Data preparation inside the training script	The vector operations the model fitting needs are native here.
scikit-learn	All six models: logistic regression, gradient boosting, Ridge	Standard, inspectable implementations. Coefficients can be exported and re-scored elsewhere, which is what the next row depends on.
Google Gemini 2.5 Flash	The AI Analyst view	Receives aggregated totals only — never raw rows — and is instructed to answer from them or say it cannot.
CSS custom properties	Three themes from a single stylesheet	Theme values are written to the document root, so one stylesheet covers all three instead of three sets of classes.
Vercel	Hosting and deployment	Builds directly from the repository. API routes become serverless functions.

The decision worth explaining. Models are **trained in Python** but **scored in TypeScript**. The training script writes coefficients, scaler parameters and category reference levels to one JSON file; the web application reads that file and computes predictions itself. So no Python process runs in production, the interactive scorer responds with no network call, and hosting stays a static Node deployment. The training script verifies its own export against `sklearn.predict_proba` to within 1e-9 on held-out rows before writing the file, so the two paths cannot drift.

4 · How data flows



Three rules that hold it together

One simulation produces both the reporting and the training data

The generator simulates the campaign once, then writes both the aggregated reporting levels and the session-level training sample from that same pass. The dashboard totals and the model training set therefore cannot disagree with each other.

No view does its own arithmetic

All derivation lives in `lib/data.ts`. Ratios are computed from summed numerators and denominators, never averaged from per-day ratios — because the mean of daily conversion rates is not the conversion rate, and that mistake produces numbers that look plausible and are wrong.

Dates are ISO everywhere

Every date is `YYYY-MM-DD` from the moment it is loaded, so sorting text sorts chronologically. Loaders normalise any other format on the way in. In `DD/MM/YYYY` form, text sorting puts 9 August after 18 August, and every trend chart in the application would be silently wrong.

The eight endpoints

ENDPOINT	RETURNS	SIZE
<code>/api/data</code>	The full funnel table, packed	2.61 MB
<code>/api/meta</code>	Meta rows: campaign × ad set × creative × city	1.14 MB
<code>/api/google</code>	Google rows: campaign × city	0.37 MB
<code>/api/linkedin</code>	LinkedIn rows: campaign × audience × city	0.12 MB
<code>/api/print</code>	Print insertions	0.01 MB
<code>/api/ml</code>	All six trained models	0.12 MB
<code>/api/quality</code>	Six data-quality checks, run live	0.01 MB
<code>/api/chat</code>	AI Analyst answer POST	—

5 • The dataset

The data is simulated. `scripts/generate.mjs` runs a seeded simulation of an advertising auction, a registration funnel and a ticket price ladder, then aggregates the outcome into 22 reporting levels and draws an 80,000-row session sample for model training. Seed `20260823`, so rebuilding reproduces the identical dataset.

FILE	ROWS	ONE ROW IS
<code>funnel_daily.csv</code>	76,453	one day × one reporting level × one entity
<code>meta_ads_daily.csv</code>	23,892	day × campaign × ad set × creative × city
<code>google_ads_daily.csv</code>	9,504	day × campaign × city
<code>linkedin_ads_daily.csv</code>	3,480	day × campaign × audience × city
<code>print_ads_daily.csv</code>	128	one newspaper insertion
<code>sessions.csv</code>	79,996	one website session, with its outcome

What the season looks like

channel	spend L	mix%	landed	regs	GTV Cr	CPC	CAC	ROAS
Meta	124.0	45.4	9,92,760	18,678	3.43	12.5	664	2.77x
Google	107.2	39.3	8,55,690	26,580	4.94	12.5	403	4.61x
Print	22.2	8.1	32,006	268	0.05	69.3	8270	0.23x
LinkedIn	19.7	7.2	25,250	352	0.07	77.8	5583	0.34x
Organic	-	-	5,03,381	15,361	2.85	-	-	-
Email	-	-	1,65,665	6,621	1.22	-	-	-
Referral	-	-	1,14,991	2,824	0.53	-	-	-
Affiliate	-	-	1,01,596	1,641	0.31	-	-	-

landed 27,91,339 -> leads 3,24,087 -> pay 1,26,651 -> registrations 72,325
 GTV Rs 13.40 Cr on Rs 2.73 Cr paid spend average order value Rs 1,853
 funnel 11.6% -> 39.1% -> 57.1% landed to registration 2.59%
 blended ROAS 4.91x paid-only ROAS 3.11x

What the simulation models

- Weekly seasonality — weekends browse, Mondays are quiet
- An exponential demand ramp into race day
- Four price-rise deadlines, each producing a spike
- Creative fatigue, at a different rate per asset type
- City income skew affecting ticket choice
- An eight-rung ticket price ladder
- Advertising auction saturation
- Budget pacing drift, independent of demand

The last two are not decoration. Auction saturation raises the effective cost per thousand impressions as daily spend rises, so registrations grow like $\text{spend}^{0.8}$ rather than in a straight line. Without it, the true spend elasticity is exactly 1.0 and the response curves in section 7 would have nothing to measure. **Pacing drift** is a random walk on each channel's daily budget, independent of demand; without it, spend is a fixed function of the demand index and perfectly correlated with the trend controls, which makes the elasticity statistically unidentifiable — the regression cannot separate the effect of spending more from the effect of the calendar.

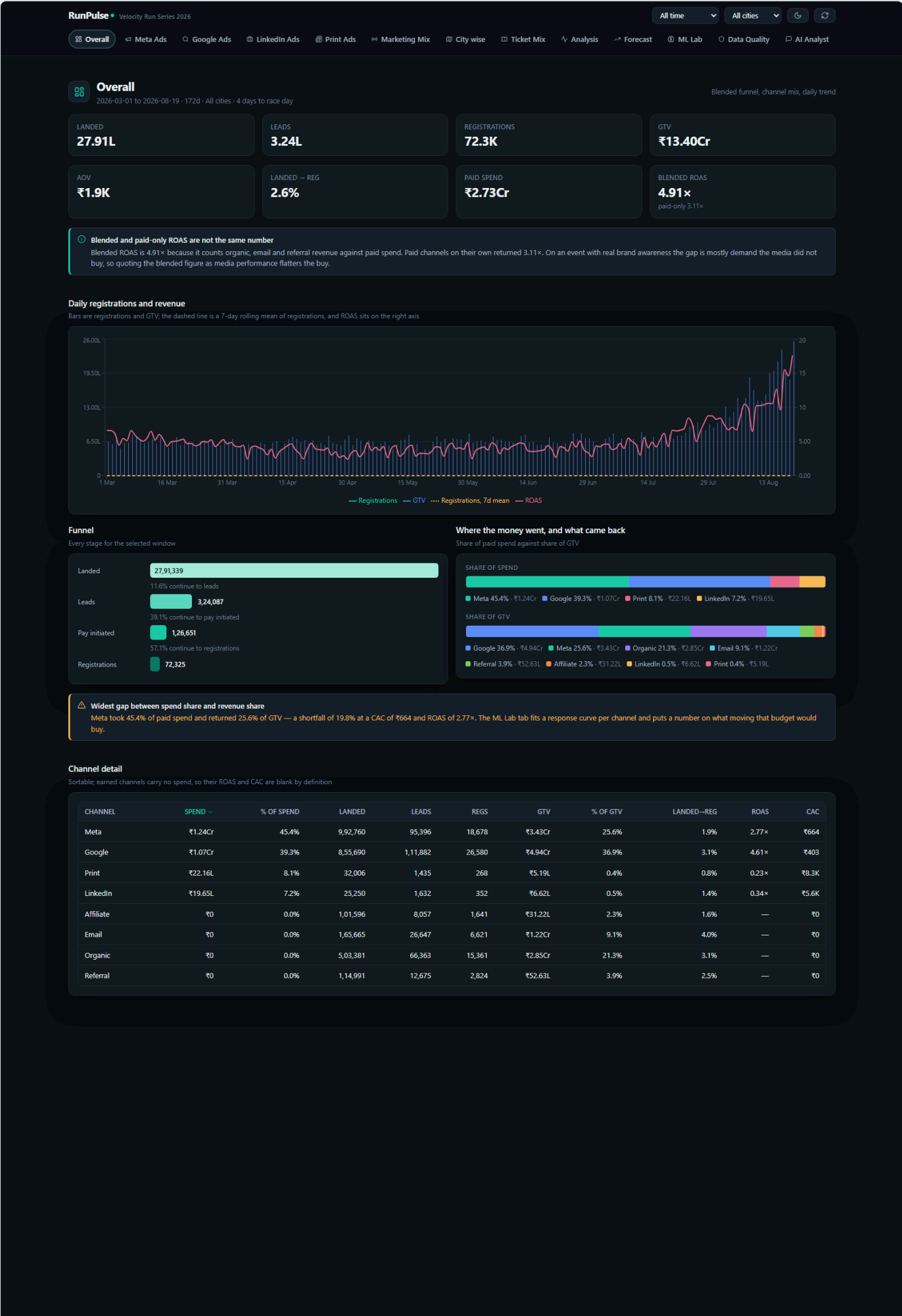
6 · The thirteen views

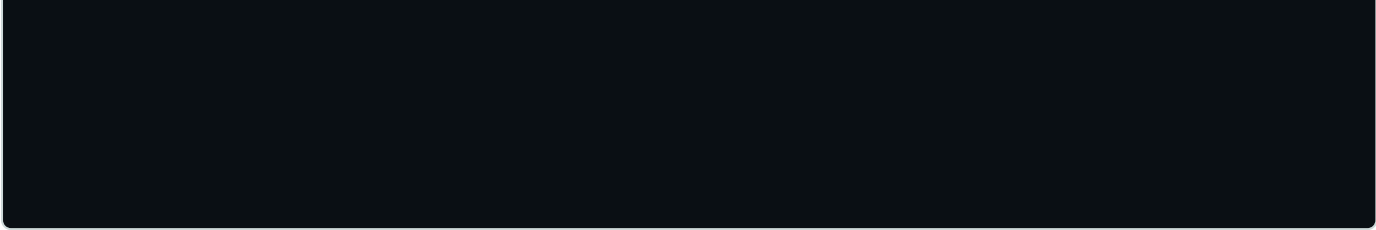
Each view, what it is for, and the one thing it is designed to make visible.

6.1 Overall

The blended picture: eight headline figures, the daily trend, the full funnel, and where spend went against where revenue came from.

Its point: blended ROAS (4.91×) counts organic, email and referral revenue against paid spend. Paid channels alone returned 3.11×. Both are shown, because quoting the blended figure as media performance flatters the buy by more than half.



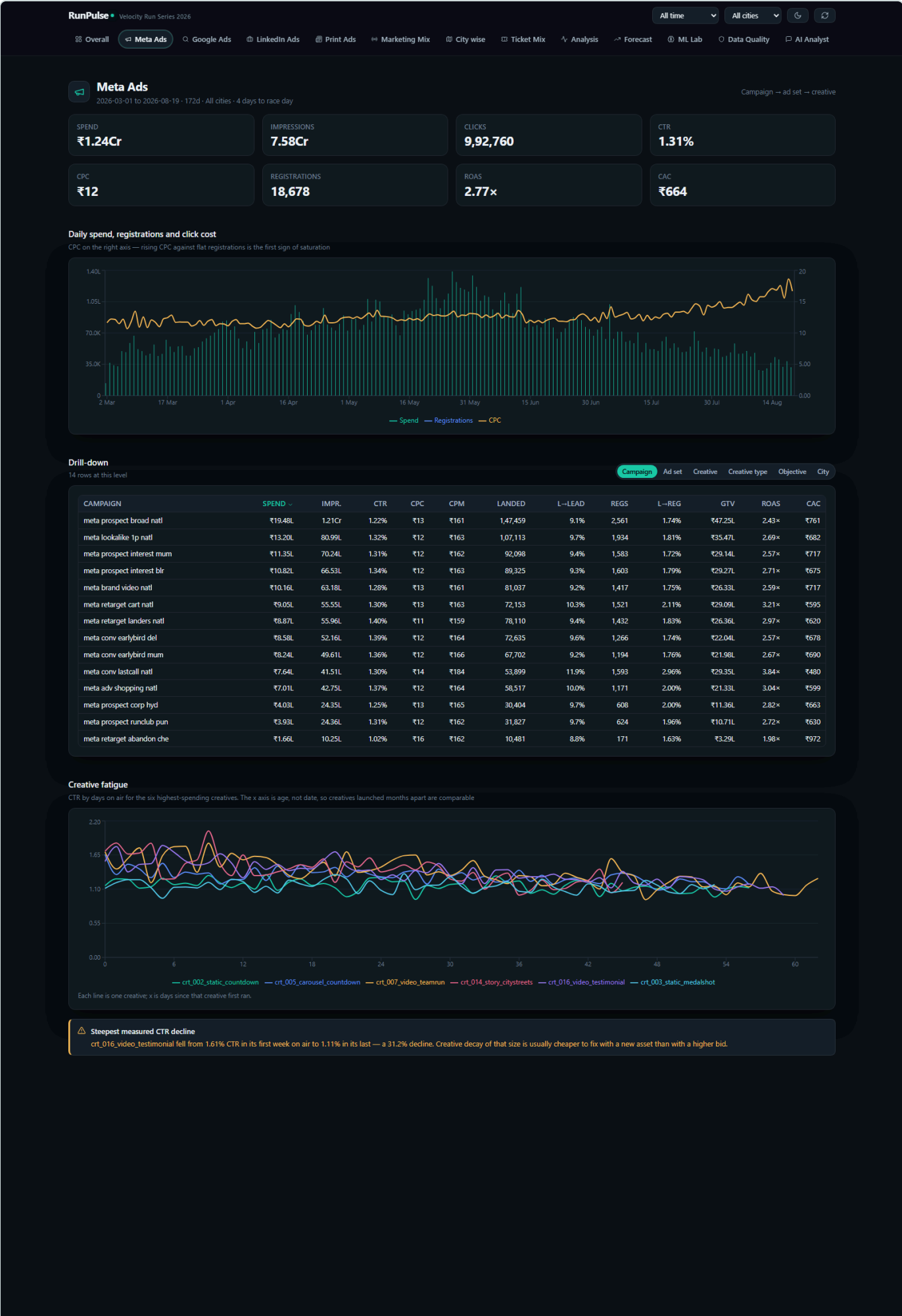


Overall — headline metrics, daily registrations and revenue, funnel, and spend against revenue share.

6.2 Meta Ads

Drill-down through campaign, ad set, creative, creative type, objective and city, with click-through rate, cost per click, cost per thousand impressions, and cost per acquisition on every row.

Its point: the creative fatigue chart plots click-through rate against *days on air* rather than against the calendar, so assets launched months apart are directly comparable and a decaying creative is obvious.



Drill-down

14 rows at this level

CampaignAd setCreativeCreative typeObjectiveCity

CAMPAIGN	SPEND	IMPR.	CTR	CPC	CPM	LANDED	L-LEAD	REGS	L-REG	GTV	ROAS	CAC
meta prospect broad natl	₹19.48L	1.21Cr	1.22%	₹13	₹161	1,47,459	9.1%	2,561	1.74%	₹47.25L	2.43×	₹761
meta lookalike 1p natl	₹13.20L	80.99L	1.32%	₹12	₹163	1,07,113	9.7%	1,934	1.81%	₹35.47L	2.69×	₹682
meta prospect interest num	₹11.35L	70.24L	1.31%	₹12	₹162	92,098	9.4%	1,583	1.72%	₹29.14L	2.57×	₹717
meta prospect interest blr	₹10.82L	66.53L	1.34%	₹12	₹163	89,325	9.3%	1,603	1.79%	₹29.27L	2.71×	₹675
meta brand video natl	₹10.16L	63.18L	1.28%	₹13	₹161	81,037	9.2%	1,417	1.75%	₹26.33L	2.59×	₹717
meta retarget cart natl	₹9.05L	55.55L	1.30%	₹13	₹163	72,153	10.3%	1,521	2.11%	₹29.09L	3.21×	₹595
meta retarget landers natl	₹8.87L	55.96L	1.40%	₹11	₹159	78,110	9.4%	1,432	1.83%	₹26.36L	2.97×	₹620
meta conv earlybird del	₹8.58L	52.16L	1.39%	₹12	₹164	72,635	9.6%	1,266	1.74%	₹22.04L	2.57×	₹678
meta conv earlybird mum	₹8.24L	49.61L	1.36%	₹12	₹166	67,702	9.2%	1,194	1.76%	₹21.98L	2.67×	₹690
meta conv lastcall natl	₹7.64L	41.51L	1.30%	₹14	₹184	53,899	11.9%	1,593	2.96%	₹29.35L	3.84×	₹480
meta adv shopping natl	₹7.01L	42.75L	1.37%	₹12	₹164	58,517	10.0%	1,171	2.00%	₹21.33L	3.04×	₹599
meta prospect corp hyd	₹4.03L	24.35L	1.25%	₹13	₹165	30,404	9.7%	608	2.00%	₹11.36L	2.82×	₹663
meta prospect runcub pun	₹3.93L	24.36L	1.31%	₹12	₹162	31,827	9.7%	624	1.96%	₹10.71L	2.72×	₹630
meta retarget abandon che	₹1.66L	10.25L	1.02%	₹16	₹162	10,481	8.8%	171	1.63%	₹3.29L	1.98×	₹972

Creative fatigue

CTR by days on air for the six highest-spending creatives. The x axis is age, not date, so creatives launched months apart are comparable

Each line is one creative; x is days since that creative first ran.

Steepest measured CTR decline

crt_016_video_testimonial fell from 1.61% CTR in its first week on air to 1.11% in its last — a 31.2% decline. Creative decay of that size is usually cheaper to fix with a new asset than with a higher bid.

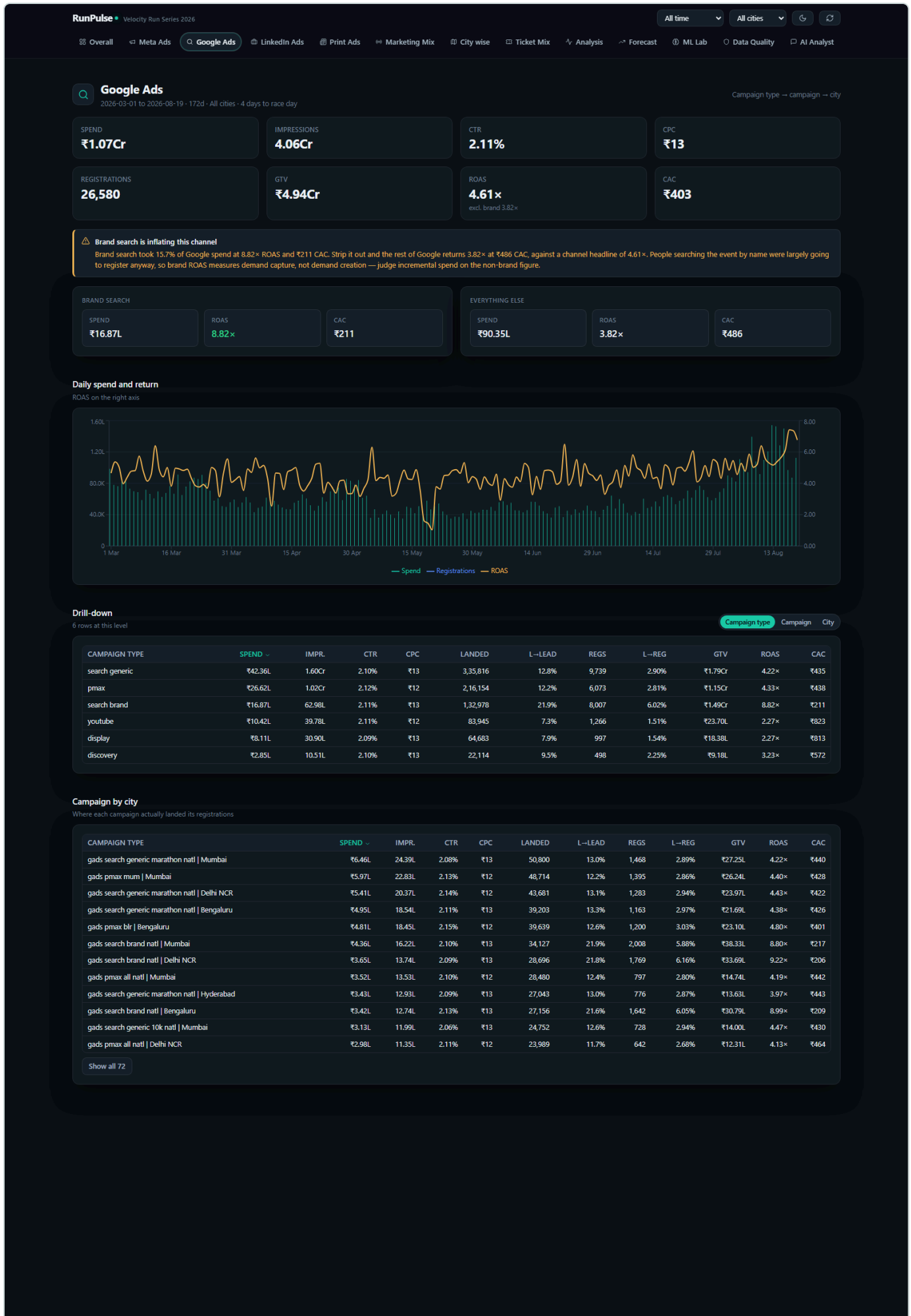


Meta Ads — spend and click cost over time, a six-dimension drill-down, and creative fatigue by asset age.

6.3 Google Ads

Performance by campaign type — brand search, generic search, Performance Max, display, YouTube, discovery.

Its point: brand search converts people who were already searching for the event by name, so its return is enormous and it lifts the whole channel average with it. The view reports Google's ROAS both with and without brand search, because only the second number should influence a decision to spend more.



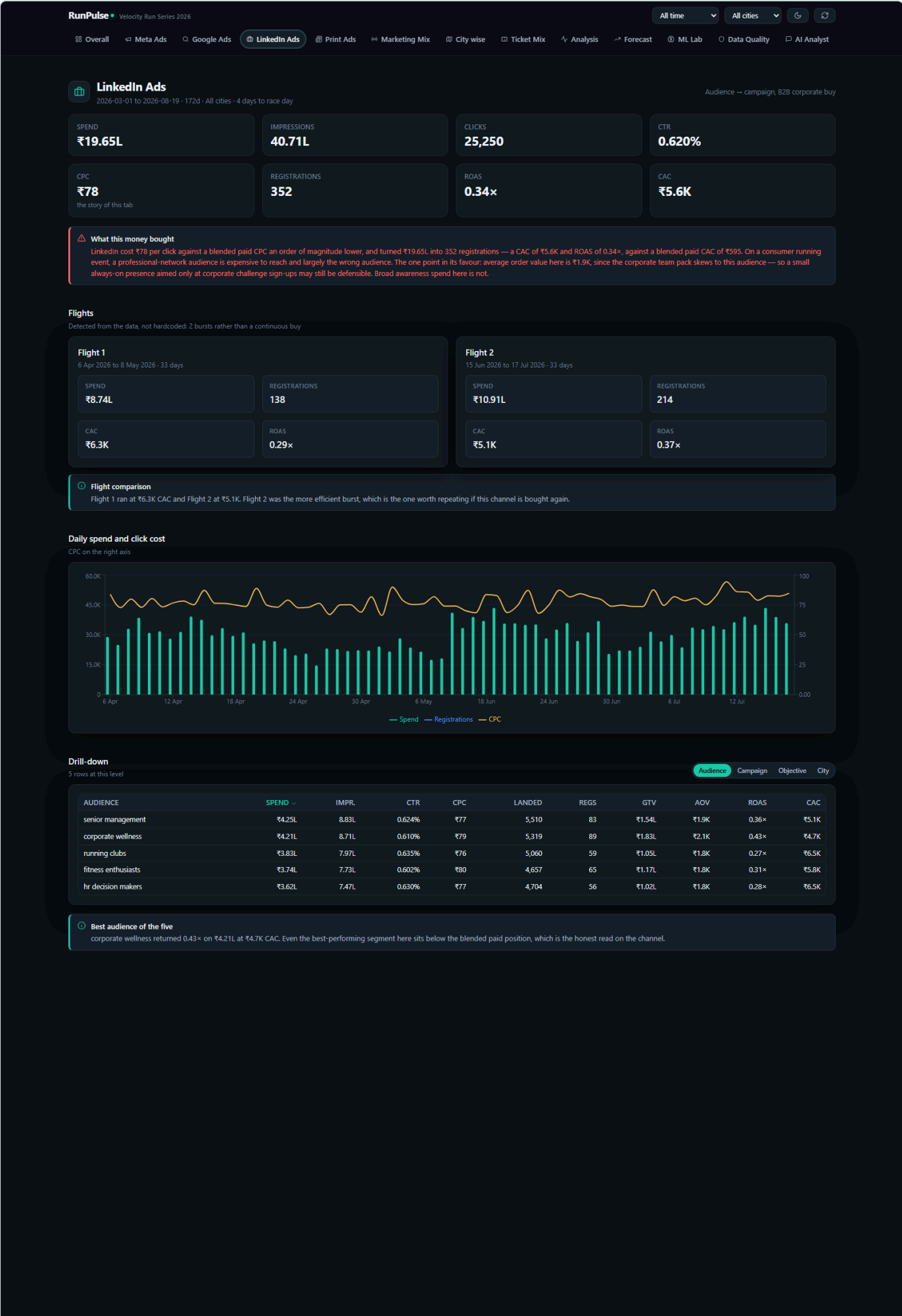


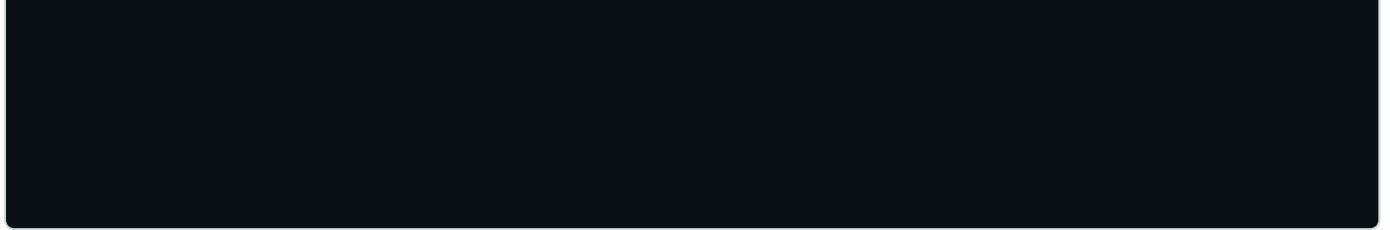
Google Ads — brand search isolated from everything else, then campaign type and campaign-by-city detail.

6.4 LinkedIn Ads

A business-to-business buy aimed at corporate team registrations, across five audience segments.

Its point: LinkedIn ran as two short bursts rather than continuously, and the view detects those flights from the data itself rather than having the dates written in. At ₹77.8 per click against roughly ₹12.5 on the other channels, it is the most expensive traffic in the account, and the view says so.





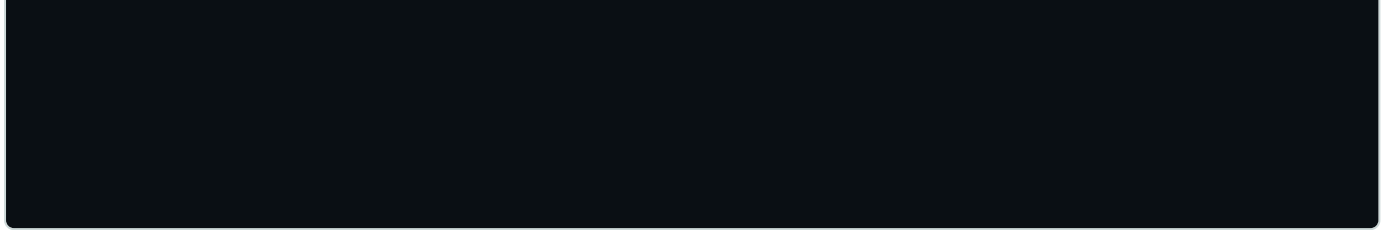
LinkedIn Ads — the two detected flights compared, then audience, campaign and city breakdowns.

6.5 Print Ads

Newspaper insertions by publication, slot size and city.

Its point: a newspaper advertisement has no click. The only tracked path from print to a registration is a QR code, which catches a tiny fraction of the people who saw it — everyone who read the ad and later searched the event by name is recorded as Google or Organic. So the view treats the tracked return as a *floor*, not a measurement, and judges the channel on cost per thousand reach instead.





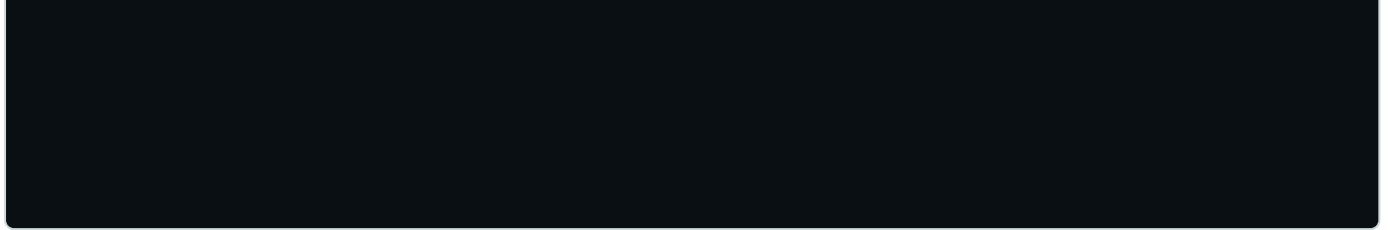
Print Ads — spend against estimated reach by publication, and the full insertion schedule.

6.6 Marketing Mix

All eight channels side by side on a metric the reader chooses, with a full comparison table.

Its point: the table is sorted by the gap between a channel's share of revenue and its share of spend. A channel taking 8% of the budget and returning 0.4% of revenue is visible immediately.



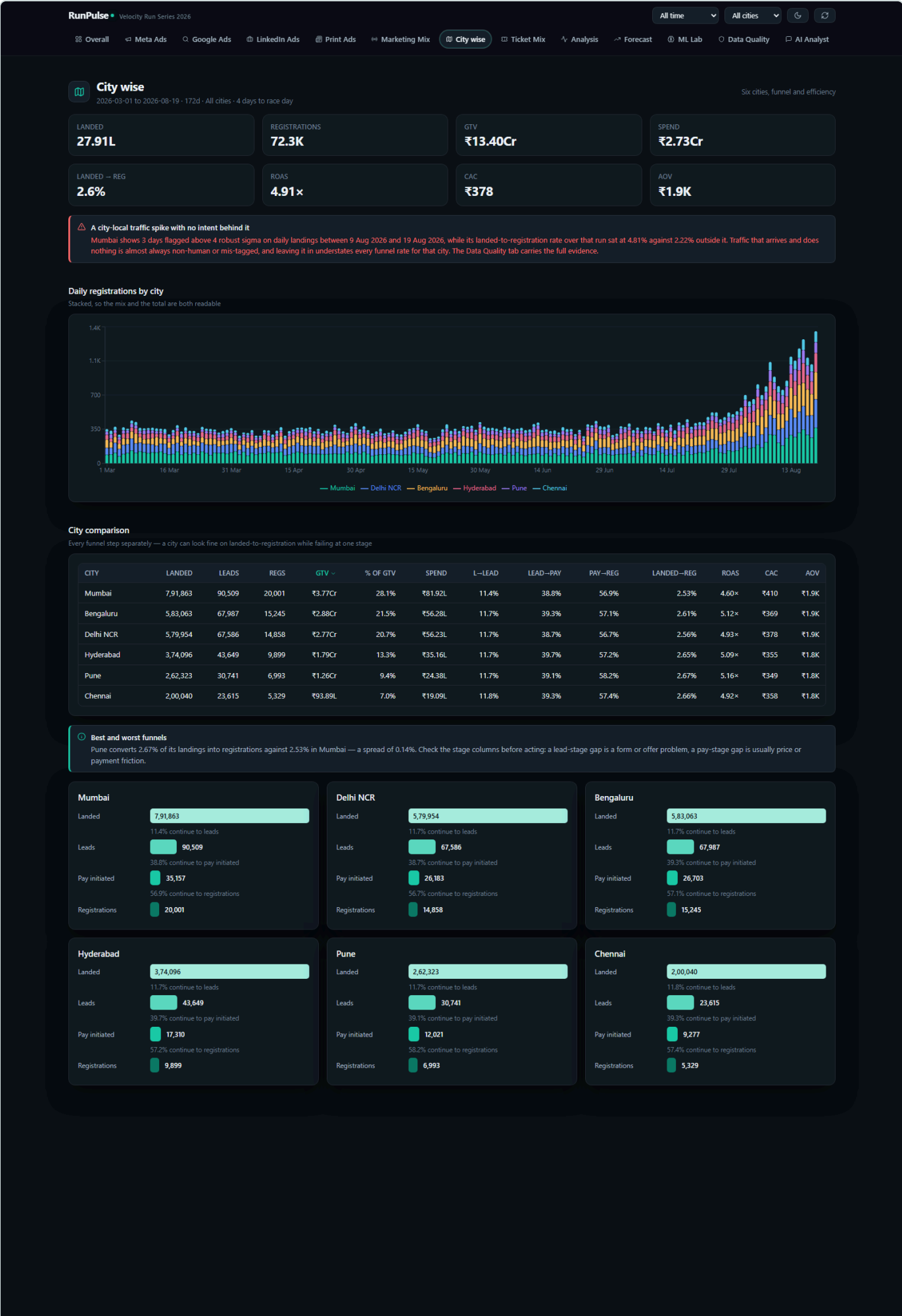


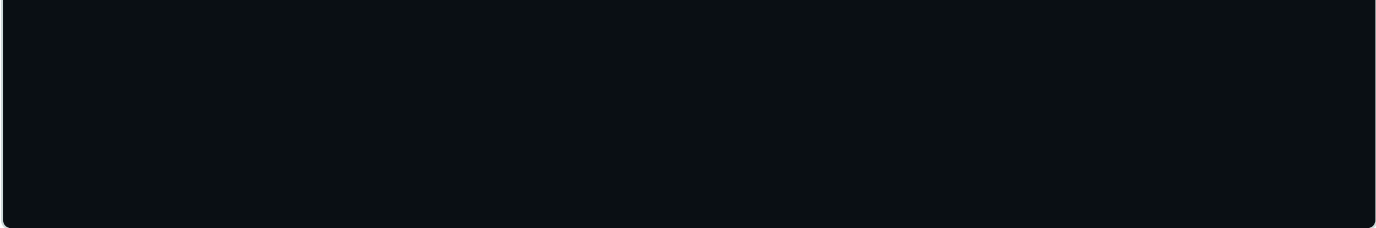
Marketing Mix — every channel on one metric, efficiency against scale, and the spend-versus-revenue share gap.

6.7 City wise

Six cities, with every funnel step reported separately rather than collapsed into one conversion rate.

Its point: a city can look acceptable on overall conversion while failing badly at one specific step. Separating the steps localises the problem — a lead-stage gap is a form or offer issue, a payment-stage gap is usually price or checkout friction.



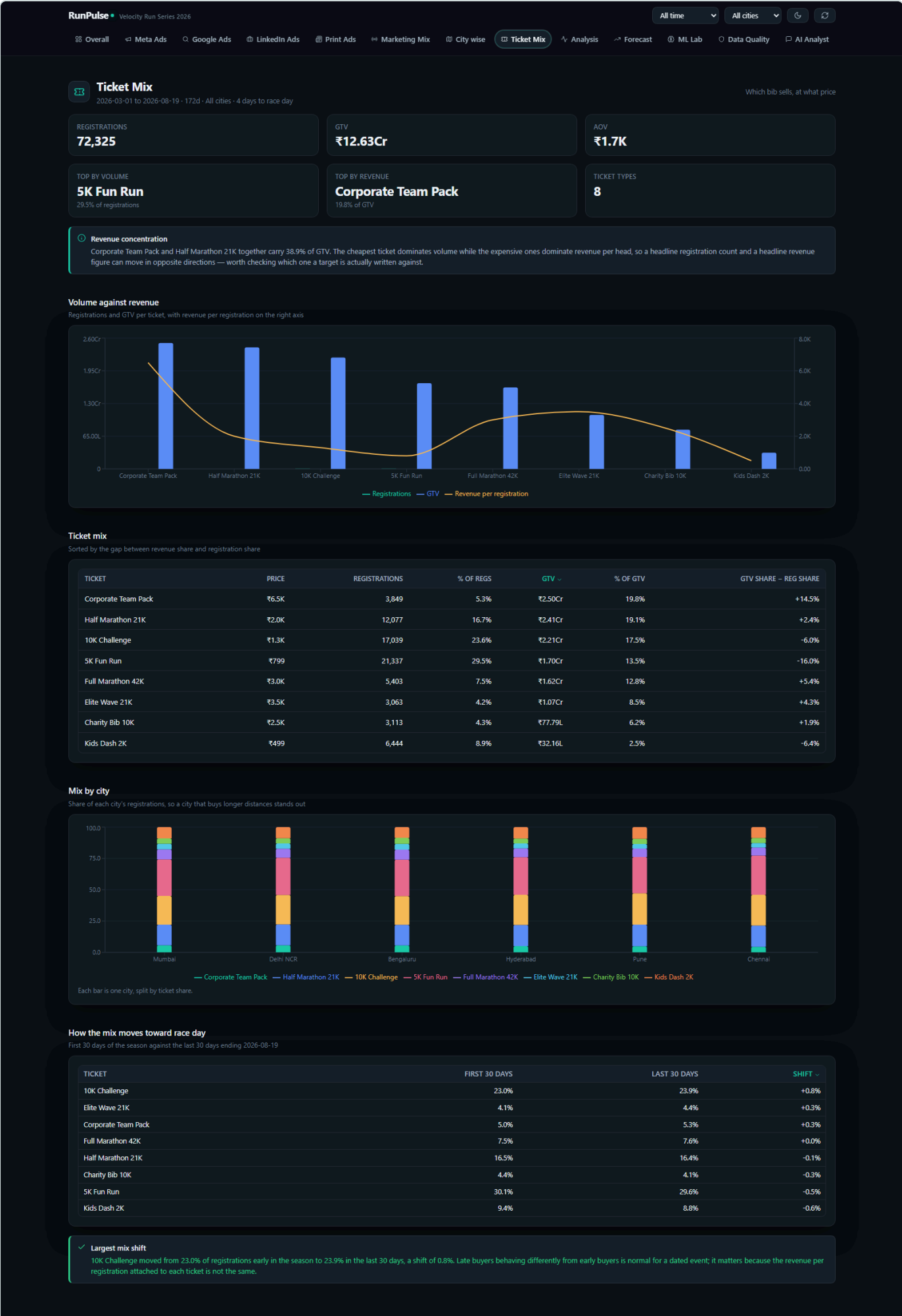


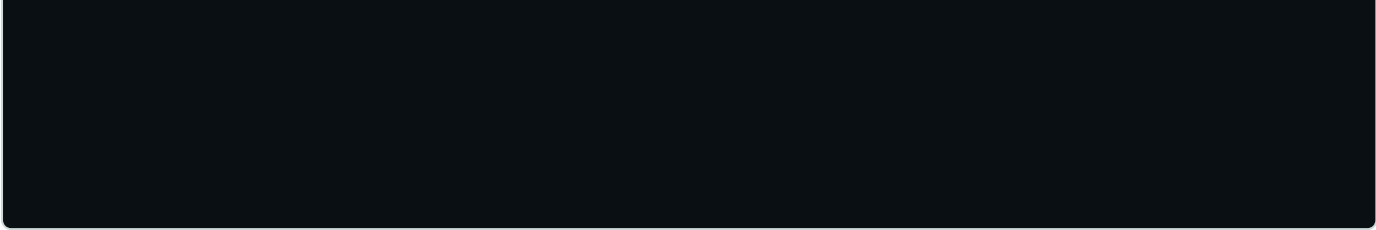
City wise — stacked daily registrations, a full comparison table, and a funnel breakdown per city.

6.8 Ticket Mix

The eight ticket types, from ₹499 to ₹6,499, by volume and by revenue.

Its point: the cheapest ticket dominates the registration count while the expensive ones dominate revenue per head, so a registrations target and a revenue target can move in opposite directions. The view also shows how the mix shifts as race day approaches.





Ticket Mix — volume against revenue per ticket, mix by city, and how the mix moves toward race day.

6.9 Analysis

The diagnostic view: biggest movers, funnel diagnostics, an anomaly scan, and an efficiency quadrant.

Its point: movers are ranked by *absolute* change, not percentage. A 400% jump on two registrations is noise, and ranking by percentage puts that at the top of the list every single week.

RunPulse

Velocity Run Series 2026

All time

All cities

Overall

Meta Ads

Google Ads

LinkedIn Ads

Print Ads

Marketing Mix

City wise

Ticket Mix

Analysis

Forecast

ML Lab

Data Quality

AI Analyst

Analysis

2026-03-01 to 2026-08-19 · 172d · All cities · 4 days to race day

Movers, flags, funnel diagnostics

ENTITIES TRACKED

8

DECLINING

0

ANOMALIES FLAGGED

191

[x] > 3.5

MEDIAN ROAS

1.55x

paid entities at this level

Channel

Meta campaigns

Google campaigns

LinkedIn

Source

City

Ticket

Registrations

GTV

Landed

Spend

Biggest rise

Google gained 26,580 conversions, from 0 to 26,580.

Movers

Ranked by absolute change, not percentage — a 400% jump on two registrations is noise, and ranking by percentage puts it top every time

ENTITY	PRIOR WINDOW	THIS WINDOW	CHANGE	%	FLAG
Google	0	26,580	+26,580	—	NA
Meta	0	18,678	+18,678	—	NA
Organic	0	15,361	+15,361	—	NA
Email	0	6,621	+6,621	—	NA
Referral	0	2,824	+2,824	—	NA
Affiliate	0	1,641	+1,641	—	NA
LinkedIn	0	352	+352	—	NA
Print	0	268	+268	—	NA

Funnel diagnostics

Each entity's step rates against the median at this level. A single negative gap localises the problem to one stage

ENTITY	LANDED	L→LEAD	VS MEDIAN	LEAD→PAY	VS MEDIAN	PAY→REG	VS MEDIAN
Email	1,65,665	16.1%	+5.8%	42.7%	+4.0%	58.2%	+1.2%
Organic	5,03,381	13.2%	+2.9%	40.6%	+1.8%	57.1%	+0.1%
Google	8,55,690	13.1%	+2.8%	40.6%	+1.9%	58.5%	+1.6%
Referral	1,14,991	11.0%	+0.7%	39.2%	+0.4%	56.9%	-0.1%
Meta	9,92,760	9.6%	-0.7%	35.4%	-3.3%	55.2%	-1.7%
Affiliate	1,01,596	7.9%	-2.4%	38.3%	-0.4%	53.2%	-3.8%
LinkedIn	25,250	6.5%	-3.9%	35.0%	-3.7%	61.5%	+4.6%
Print	32,006	4.5%	-5.8%	35.7%	-3.0%	52.2%	-4.7%

Anomaly scan

Robust z-score on the median absolute deviation. A plain mean-and-sigma score lets a large spike inflate sigma enough to hide itself

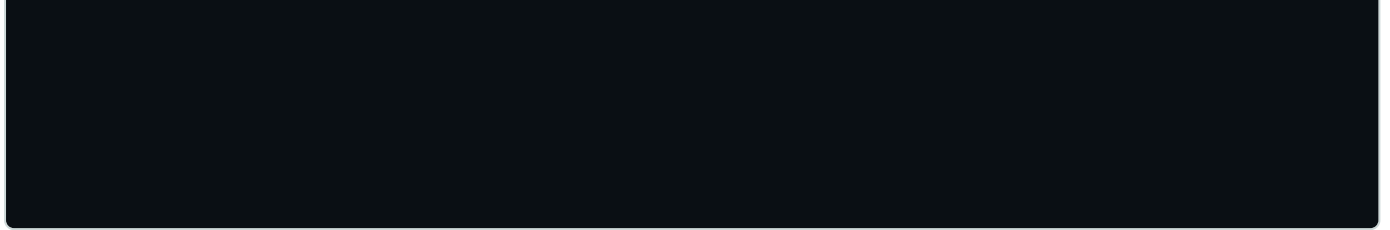
DATE	ENTITY	METRIC	VALUE	ROBUST Z
19 Aug 2026	Email	registrations	258	22.3
15 Aug 2026	Organic	registrations	463	20.0
19 Aug 2026	Organic	registrations	461	19.9
13 Aug 2026	Organic	registrations	380	15.8
16 Aug 2026	Email	registrations	189	15.6
17 Aug 2026	Organic	registrations	365	15.1
19 Aug 2026	Referral	registrations	99	14.7
14 Aug 2026	Organic	registrations	352	14.4
16 Aug 2026	Organic	registrations	348	14.2
18 Aug 2026	Email	registrations	163	13.1
18 Aug 2026	Organic	registrations	319	12.8
17 Aug 2026	Email	registrations	158	12.6

Show all 191

Efficiency quadrant

Split at the window medians: spend ₹64.69L, ROAS 1.55x

ENTITY	SPEND	ROAS	REGS	VERDICT
Meta	₹1.24Cr	2.77x	18,678	Hold — working at scale
Google	₹1.07Cr	4.61x	26,580	Hold — working at scale
Print	₹22.16L	0.23x	268	Hold — small
LinkedIn	₹19.65L	0.34x	352	Hold — small



Analysis — movers with status flags, per-entity funnel diagnostics against the median, and an anomaly scan.

6.10 Forecast

A projection of registrations, revenue, traffic and spend through to race day, with prediction intervals.

Its point: the error of a naive baseline is printed directly beside the model's own error. A forecast that cannot beat "assume next week looks like last week" is not worth acting on, and the reader should be able to see that without taking it on trust.



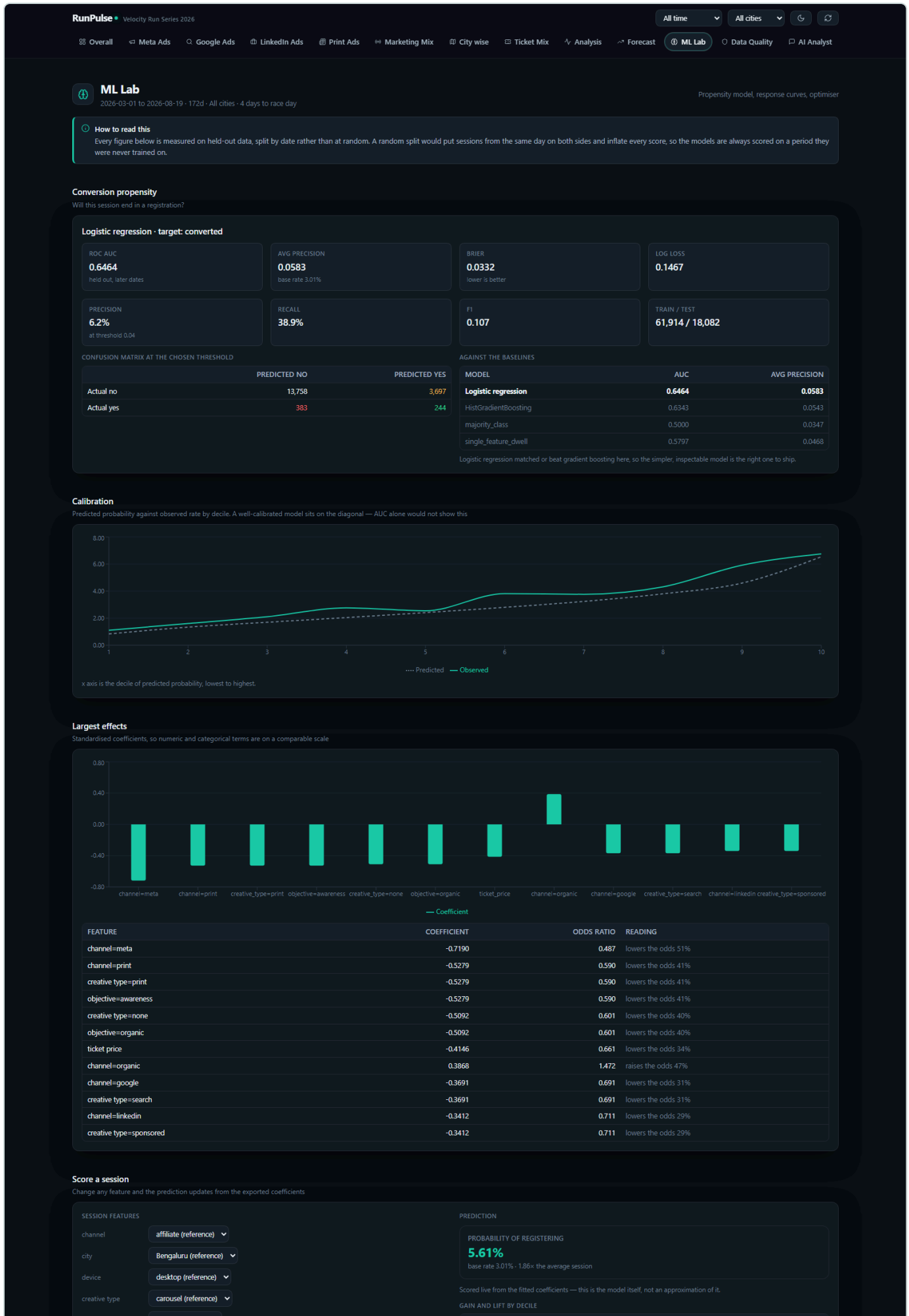


Forecast — actuals, fit and projection with a 95% band, backtest results, and a day-by-day table to race day.

6.11 ML Lab

The modelling work presented for inspection: held-out metrics, a confusion matrix, a calibration curve, the largest effects as odds ratios, a live scorer, the response curves, and the budget optimiser.

Its point: the scorer is interactive. Change the channel, city, device or session depth and the predicted probability updates immediately, computed from the exported coefficients rather than from a stored lookup.



price tier	entry (reference)		DECILE	PREDICTED	ACTUAL	LIFT	CUM. SHARE OF REGISTRATIONS	
objective	advantage (reference)		1	6.54%	6.75%	1.95×	19.5%	
days to event	86	mean 85.9	2	4.60%	5.92%	1.71×	36.5%	
session depth	6	mean 6.3	3	3.80%	4.31%	1.24×	49.0%	
dwell seconds	280	mean 279.9	4	3.24%	3.76%	1.08×	59.8%	
prior visits	1	mean 0.9	5	2.80%	3.82%	1.10×	70.8%	
ticket price	1716	mean 1716.1	6	2.41%	2.54%	0.73×	78.1%	
			7	2.04%	2.77%	0.80×	86.1%	
			8	1.69%	2.10%	0.61×	92.2%	

ML Lab — model card with baselines, calibration, the twelve largest effects as odds ratios, and the live scorer.

6.12 Data Quality

Six checks run live on every load, plus the reconciliation figures and the two known tracking faults.

Its point: every check prints what it expected next to what it measured, so a green tick can be judged instead of trusted. And the system is required to keep catching two faults that are already known — a monitor that cannot prove a catch on a fault you know about is not worth trusting on one you do not.

RunPulse

Velocity Run Series 2026

All timeAll cities

OverallMeta AdsGoogle AdsLinkedIn AdsPrint AdsMarketing MixCity wiseTicket MixAnalysisForecastML LabData QualityAI Analyst

Data Quality

2026-03-01 to 2026-08-19 · 172d · All cities · 4 days to race day

Reconciliation and tracking faults

CHECKS PASSED

5 / 6

WARNINGS

1

FAILURES

0

FAULTS DETECTED

2 / 2

How to read this

Every check can genuinely fail, and each prints what it expected alongside what it measured — so a green tick can be judged rather than taken on trust.

Checks

Run live against the CSVs on every request

Channel rows sum to the overall level

Every day reconciles across landed, leads, registrations and GTV.

expected: 0 mismatched day-metrics (GTV tolerance 2)measured: 0 mismatched

PASS

City levels roll up to their parent, and every parent string resolves

All seven city levels reconcile exactly to their parents.

expected: 0 orphan parents, 0 registration mismatchesmeasured: 0 orphans, 0 mismatches

PASS

Channel reporting reconciles to the funnel table

Largest gap is Google spend at -0.0002%. Money is stored rounded to whole rupees, so a residual of a few thousandths of a percent is expected there; counts must agree exactly.

expected: within 0.5% on every metricmeasured: worst -0.000%

PASS

Every day in the season is present and nothing falls outside it

172 consecutive days from 2026-03-01 to 2026-08-19.

expected: 172 days, 0 outsidemeasured: 172 days, 0 outside

PASS

No negatives, and each funnel stage is no larger than the one before it

All 76,453 rows non-negative; every day orders registrations <= pay <= leads <= landed.

expected: 0 negatives, 0 ordering breaksmeasured: 0 negatives, 0 ordering breaks

PASS

Known tracking faults are still being detected

These are deliberate. They exist so the pipeline has something real to catch; a status of warn is the correct resting state, not a problem to fix.

expected: 2 of 2 detectedmeasured: 2 of 2 detected

WARN

Reconciliation

Each channel against the funnel table, with the tolerance shown rather than assumed

CHANNEL	METRIC	CHANNEL REPORT	FUNNEL TABLE	DIFFERENCE	%	TOLERANCE
Meta	spend	₹1.24Cr	₹1.24Cr	-17	-0.000%	0.5%
Meta	landed	9,92,760	9,92,760	0	0.000%	0.5%
Meta	conversions	18,678	18,678	0	0.000%	0.5%
Meta	gtv	₹3.43Cr	₹3.43Cr	0	0.000%	0.5%
Google	spend	₹1.07Cr	₹1.07Cr	-18	-0.000%	0.5%
Google	landed	8,55,690	8,55,690	0	0.000%	0.5%
Google	conversions	26,580	26,580	0	0.000%	0.5%
Google	gtv	₹4.94Cr	₹4.94Cr	0	0.000%	0.5%
LinkedIn	spend	₹19.65L	₹19.65L	-3	-0.000%	0.5%
LinkedIn	landed	25,250	25,250	0	0.000%	0.5%
LinkedIn	conversions	352	352	0	0.000%	0.5%
LinkedIn	gtv	₹6.62L	₹6.62L	0	0.000%	0.5%
Print	spend	₹22.16L	₹22.16L	+0	0.000%	0.5%
Print	landed	32,006	32,006	0	0.000%	0.5%
Print	conversions	268	268	0	0.000%	0.5%
Print	gtv	₹5.19L	₹5.19L	0	0.000%	0.5%

Money is stored rounded to whole rupees, so a small residual there is expected. Counts must agree exactly.

Known tracking faults

Two faults in this season, and the measured evidence for each

Lead events stopped firing on Google traffic

18–20 May 2026

Google landed—lead fell to 3.8% against a season median of 12.4%, with landings unaffected — a tracking failure, not a media one.

DETECTED

Crawler inflated organic landings in Hyderabad

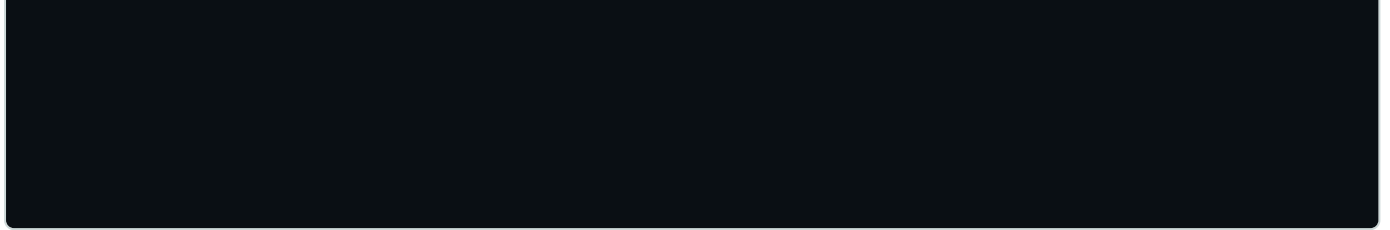
25 Jun – 2 Jul 2026

Median daily landings rose from 365 to 912 (2.50x) while landed—registration fell from 2.75% to 0.34% — volume with no intent behind it.

DETECTED

What this proves

Both faults are surfaced by the checks above and by the anomaly scan on the Analysis tab, with the measured numbers shown rather than asserted. A monitor that cannot prove a catch on a fault you already know about is not worth trusting on one you do not.



Data Quality — six checks with expected against measured, reconciliation with tolerances, and the two known faults.

6.13 AI Analyst

Natural-language questions answered from the season's aggregate figures.

Its point: the model is given totals, rates and top-ten lists — never raw rows — and is instructed to answer only from those and to say plainly when the answer is not there. It cannot invent a number it was not given, and it cannot answer about one individual session.

RunPulse

Velocity Run Series 2026

All timeAll cities

OverallMeta AdsGoogle AdsLinkedIn AdsPrint AdsMarketing MixCity wiseTicket MixAnalysisForecastML LabData QualityAI Analyst

AI Analyst

2026-03-01 to 2026-08-19 · 172d · All cities · 4 days to race day

Ask the dataset a question

What this can and cannot answer

The model is given the season aggregates — channel, city, campaign, ticket and funnel totals, the last fortnight day by day, and the model results — and is instructed to answer only from those and to say so when the answer is not there. It is not querying the raw rows, so it cannot tell you about one specific session.

Ask the dataset

Or start with one of these

Which channel should I cut, and where should the money go?

Why is Google's ROAS so much higher than Meta's?

What happened to leads on 18 May?

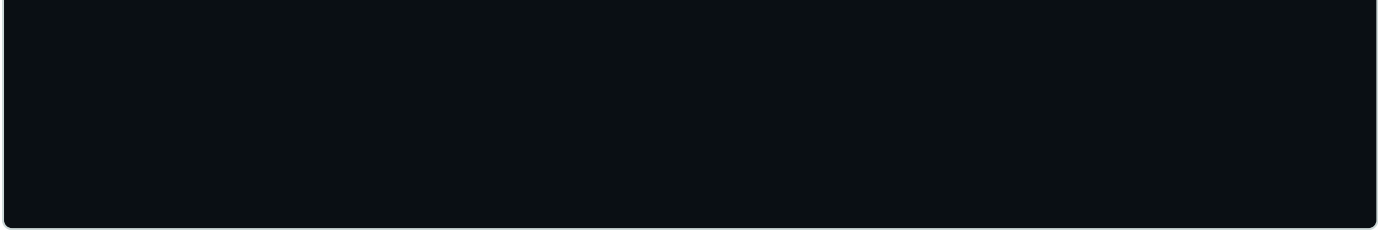
Which city has the worst funnel, and at which step?

What will final registrations be on race day?

Is print worth the money?

Ask about channels, cities, campaigns, tickets or the forecast

Send



AI Analyst — suggested starting questions, and answers grounded in the supplied aggregates.

7 • The models

Six models, fitted by `ml/train.py`. Every number below is measured on data the model never saw during training.

7.1 and 7.2 — Will this visitor register? Will they submit a lead?

Two logistic regressions over 13 features: seven numeric (days to race day, session depth, time on page, prior visits, ticket price, returning visitor, weekend) and six categorical (channel, city, device, creative type, price tier, campaign objective).

TARGET	ROC AUC	AVG PRECISION	BRIER	GRADIENT BOOSTING
Registered (3.0% base rate)	0.6464	0.0583	0.0332	0.6343
Submitted a lead	0.6173	0.2002	—	0.5981

The split is by date, not random

Training uses the first 80% of the season and testing the last 20%. A random split would place sessions from the same day on both sides; since demand on a given day drives conversion on that day, the model would effectively be told the answer, and the reported accuracy would be inflated.

Class weighting was deliberately removed

Balancing a 3% positive rate is the textbook reflex and it was wrong here. It shifts the intercept so far that the model's average predicted probability becomes 48% against an actual rate of 3.5% — the ranking stays fine, but every percentage the interface displays becomes meaningless, and the Brier score degrades from 0.033 to 0.244. Removing it costs 0.0016 of AUC and produces calibrated output. The calibration curve in ML Lab now tracks the diagonal: the lowest decile predicts 0.83% against an observed 1.11%, the highest predicts 6.54% against 6.75%.

7.3 — How does the season finish?

Ridge regression on `log1p` of each series, with a linear trend, a race-day proximity ramp, Fourier terms at a weekly period, day-of-week indicators, and the values from one day and seven days earlier. Multi-step predictions are recursive, so the model builds on its own output rather than on actual values it will not have.

SERIES	MODEL ERROR	NAIVE BASELINE	VERDICT
Registrations	16.1%	20.0%	beats the baseline
Revenue	16.9%	20.8%	beats the baseline
Spend	15.4%	19.0%	beats the baseline
Traffic	10.4%	10.2%	a tie, reported as one

Error is mean absolute percentage error over five rolling-origin folds with a seven-day horizon, refitting at each origin. The seven-day lag term is included on purpose: the naive baseline is a seven-day lag, so a model that leaves it out can only beat that baseline by luck.

7.4 — What does the next rupee buy?

A curve of the form `registrations = a × spend^b` fitted per channel, holding trend, race-day proximity and day of week constant.

CHANNEL	ELASTICITY	FIT QUALITY	AVERAGE CPA	MARGINAL CPA
Meta	0.933	0.889	₹471	₹505
Google	0.820	0.797	₹316	₹385
LinkedIn	0.950	0.388	₹4,607	₹4,850
Print	0.950	0.645	₹6,802	₹7,160

Why the demand controls matter. Fitting spend against registrations on its own returns an elasticity of **1.88** for Google — meaning each additional rupee buys *more* than the one before it, which no advertising auction can do. It is a textbook omitted-variable problem: budgets rise as the event approaches and so does organic demand, so the regression credits the spending for what the calendar was doing. Acting on that number would have moved the entire budget into one channel for an imaginary gain.

Marginal, not average. Average cost per acquisition describes money already spent. The cost of the *next* registration is the only figure a budget decision can use, and on a curve with diminishing returns the two are different.

7.5 — Where should the budget go?

The optimiser holds total daily spend fixed and moves it ₹1,000 at a time toward whichever channel currently has the lowest marginal cost per registration. Each channel is floored at 20% and capped at 250% of its current spend, because bidding and creative both need time to re-learn and a channel taken to zero loses its accumulated audience data. A curve fitting worse than 0.25 is not permitted to receive additional budget at all.

Result: +38.1% more registrations per day at the same total spend — Print down 79%, Meta down 79%, LinkedIn down 80%, Google up 89%.

7.6 — What broke, and when?

A robust outlier score using the median absolute deviation, computed on each day's ratio to a 31-day centred rolling median.

- **Median absolute deviation rather than standard deviation**, because a single large spike inflates the standard deviation enough to push its own score back under the threshold — which is exactly how bad data survives in a dataset for weeks.
- **Detrended**, because this season ramps steeply into race day, and an untreated score on a rising series simply flags the rise. The first version of this detector found race week and missed a genuine three-day tracking failure in May entirely.
- **A 31-day window**, sized against the shortest fault worth catching. Inside a 15-day window, an eight-day incident is more than half the baseline and drags the median up along with itself, hiding the fault.

8 • Data quality

Six checks run on every request. Each reports what it expected alongside what it actually measured.

CHECK	WHAT IT PROVES	RESULT
Channel totals sum to the overall total, every day, on four metrics	No traffic or revenue is double counted or lost between levels	pass · 0 mismatches
All seven city levels roll up to their parent, and every parent key resolves	No drill-down is silently empty	pass · 0 orphans
Each channel's own reporting reconciles to the funnel table	Two independent paths to the same number agree	pass · worst 0.00016%
Every day of the season present, nothing outside it	No missing days quietly depressing a total	pass · 172 days
No negatives; registrations ≤ payments ≤ leads ≤ visits, daily	The funnel cannot widen as it descends	pass
Two known tracking faults still detected	The monitoring still works	2 of 2 found

The second check is subtler than it looks. Every child row names its parent by a text key. If that key stops matching an entity at the level above — a renamed campaign, a changed separator — the child rows simply stop appearing under their parent. Nothing raises an error, no total changes, and a drill-down that used to show twelve campaigns now shows none. It is the kind of fault that survives for months because everything looks fine.

The two known faults

FAULT	WINDOW	MEASURED EVIDENCE
Lead tracking stopped firing on Google traffic	18–20 May	Visit-to-lead rate fell to 3.8% against a season median of 12.4%, while visits held steady. Traffic arriving normally but no leads recorded is a tracking failure, not a media problem — and the distinction decides whether anyone should touch the campaign.
An automated crawler inflated organic traffic in Hyderabad	25 Jun – 2 Jul	Median daily visits rose from 365 to 912 (2.50×) while the visit-to-registration rate collapsed from 2.75% to 0.34% . Volume with no intent behind it, which understates every conversion rate for that city while it is included.

9 • Engineering decisions

Four problems that shaped the implementation, and what was done about each.

9.1 The API responses were too large to deploy

The funnel table is 76,453 rows that repeat the same 172 dates, 22 level names and 667 entity names over and over. Serialised as a list of objects, that is **17.33 MB** — and a serverless function may return at most 4.5 MB, so the deployed application would have failed on its very first request.

Solution. Each repeated text value is stored once in a dictionary and referenced by position, and each row is sent as a plain list of numbers rather than a labelled object. The browser rebuilds the original rows on arrival. No information is lost.

ENDPOINT	BEFORE	AFTER	REDUCTION
/api/data	17.33 MB	2.61 MB	85%
/api/meta	8.48 MB	1.14 MB	87%
/api/google	2.17 MB	0.37 MB	83%
/api/linkedin	0.86 MB	0.12 MB	86%

9.2 The response curves reported impossible economics

Covered in section 7.4. Fitting spend against registrations without demand controls returned an elasticity above 1 for Google, implying increasing returns to advertising. Adding trend, race-day proximity and day-of-week controls brought every channel into the plausible 0.82–0.95 range.

9.3 The anomaly detector found the wrong things

Covered in section 7.6. The first version scored raw daily values and flagged race week — the campaign working as intended — while missing a real three-day tracking failure. Detrending against a rolling median, and widening the window to 31 days, fixed both.

9.4 The forecast lost to doing nothing

The first specification had no race-day ramp and no lag terms, and scored 35.8% error against a naive baseline's 17.5%. Registrations for a dated event do not grow in a straight line; they surge in the final fortnight. Adding an explicit proximity ramp and the seven-day lag brought the model ahead of the baseline on three of four series.

The common thread: in all four cases the first implementation produced output that looked plausible. The response curve gave a confident number, the detector returned a full list of anomalies, the forecast drew a smooth line. Each was caught by checking the result against something external — an economic constraint, a fault whose dates were already known, a naive baseline — rather than by reading the code again.

10 · File map

PATH	RESPONSIBILITY
<code>app/page.tsx</code>	Owns all application state — date range, city filter, active view — and builds the single context object every view receives.
<code>app/layout.tsx</code>	Document shell, metadata, theme provider.
<code>app/globals.css</code>	The entire design system, written against CSS variables so three themes work from one file.
<code>app/api/*/route.ts</code>	Eight server endpoints. None throws: a failure returns a readable message so the interface degrades instead of blanking.
<code>lib/types.ts</code>	Every shared shape. The contract both the server and the views compile against.
<code>lib/csv.ts</code>	Server-only loaders. Each file is parsed once and cached for the process lifetime.
<code>lib/data.ts</code>	All derivation: totals, ratios, slicing by level and city, movers, rolling means, robust outlier scores.
<code>lib/pack.ts</code>	The compact wire format from section 9.1. Encoder and decoder live together so they cannot drift apart.
<code>lib/ml.ts</code>	Scores the exported model coefficients in the browser. Also the response-curve maths.
<code>lib/quality.ts</code>	The six data-quality checks.
<code>lib/tabkit.ts</code>	Helpers shared across views so thirteen files do not each reimplement the same aggregation.
<code>lib/theme.tsx</code>	Theme state, persisted, written to the document root as CSS variables.
<code>lib/format.ts</code>	Indian-numbering formatters. Every displayed number passes through one of these.
<code>components/</code>	Ten shared components — table, chart, funnel, metric cards, and so on.
<code>components/tabs/</code>	One file per view, thirteen in total.
<code>scripts/generate.mjs</code>	The simulation. Produces every CSV.
<code>scripts/config.mjs</code>	The world definition: cities, channels, campaigns, publications, tickets.
<code>ml/train.py</code>	Fits all six models and exports them as one JSON file.
<code>ml/models/bundle.json</code>	The exported models the application reads.

Deployment

The repository is connected to Vercel. A push to `main` triggers a build; the API routes become serverless functions and the interface is served from the edge. Environment values, including the Gemini key, are stored encrypted in the hosting project and are never committed to the repository.

11 • Limitations

What this project does not establish, stated plainly.

The response curves show association, not causation

They are fitted on spending that was chosen by a marketing team, not assigned by an experiment. Demand controls remove the most obvious confounder but cannot remove all of them. The optimiser's +38% is what the fitted relationship implies if it holds, not a promised outcome. Establishing causality would require deliberately randomised spend — a geographic holdout test or a budget split experiment.

Attribution is last-touch throughout

A registration is credited to the last channel the visitor came through. Print's near-zero measured return is a direct consequence of that rule, not a measurement of what print achieved: a reader who saw a newspaper advertisement and later searched the event by name is recorded as Google. Any channel bought for awareness is systematically undercounted by this method.

The propensity model is a ranking tool, not a decision tool

An AUC of 0.6464 on a 3% base rate is genuine signal, and the calibration is good. It is not a strong classifier. It is suitable for ordering audiences by likelihood — the top decile converts about twice the base rate — and unsuitable for a confident judgement about any one individual visitor.

The forecast interval describes past error, not future risk

The band is derived from how wrong this model has been historically on this series. It does not account for anything that has not happened before — a competing event, a weather cancellation, a payment gateway failing during the final week.

The data is simulated

It is realistic because the generative process is explicit and the resulting funnel rates, click costs and channel economics fall in plausible ranges. But nothing here validates the simulation against a real advertising account. The engineering, the modelling method and the quality checks are what the project demonstrates.

RunPulse Analytics · [runpulse-bhargavi-aitwade-mit-wpu-capstone.vercel.app](#) · [github.com/shashanx52/runpulse-analytics](#)